

A Cross-Layer Mutual-Information Extension via Matrix-Based Rényi Entropy

thesis-IV working notes (HyMeKo)

2026-04-26

Abstract

We formalise the cross-layer mutual-information regulariser proposed as Path F of the universality programme. Using the matrix-based Rényi- α entropy framework of Sanchez-Giraldo, Rao, and Principe, we show that four extensions of the existing scalar-spectral-entropy regulariser (temporal-conditional, cross-layer mutual information, joint view entropy, and matrix-based information bottleneck) are all instances of a single primitive: the joint of two trace-normalised activation Gram matrices via Hadamard product. The $\alpha = 2$ collision-entropy form admits a closed-form, eigendecomposition-free, batch-quadratic estimator and is differentiable through standard autograd. Three falsifiable predictions for the phase-8 empirical test conclude.

1 Motivation

The nine entropy regularisers tested in phases 1–7b all use *marginal* spectral entropy at a single timestep. Specifically, for layer weights W with adjacency A and normalised-Laplacian eigenvalues $\{\lambda_i\}$,

$$H_{\text{marg}}(A) = - \sum p_i \log p_i, \quad p_i = \lambda_i / \sum_j \lambda_j.$$

This captures *how spread out* the weight spectrum is, but never:

- the **joint** distribution across layers $H(W_i, W_j)$,
- the **conditional** distribution across time $H(W_t \mid W_{t-1})$,
- or the **mutual information** between any of those.

Phases 6 and 7 demonstrated that a single normalisation (`scalar_entropy_normalized`, Path B) and target-entropy formulation (`entropy_target`, Path A) achieve universality on a 4-dataset stress matrix (spirals, circles, MNIST plain MLP, CapsMLP MNIST), but neither addresses the structural hypothesis that *layers may be redundant with each other* and that decorrelating them might be a stronger universality lever than tuning a single scalar target.

This note develops a single quantity — the matrix-based Rényi- α mutual information between activation Gram matrices of two layers — which subsumes the four extension paths under one definition.

2 Setup and notation

Let $M : \mathbb{R}^d \rightarrow \mathbb{R}^c$ be a feedforward network with L layers parameterised by weights $W_1, \dots, W_L$. Let $X \in \mathbb{R}^{B \times d}$ be a batch of B input examples. Define the per-layer activation tensor

$$a_l = \varphi_l(W_l \varphi_{l-1}(\dots W_1 X^\top))^\top \in \mathbb{R}^{B \times d_l},$$

i.e. the post-non-linearity output of layer l evaluated on X .

Construct the **sample-side Gram matrix** of layer l ,

$$K_l(X) = a_l a_l^\top \in \mathbb{R}^{B \times B}, \quad (1)$$

normalised so trace is 1,

$$\hat{K}_l = K_l / \text{tr}(K_l), \quad \text{tr}(\hat{K}_l) = 1. \quad (2)$$

$\hat{K}_l$ is symmetric positive semi-definite and trace-1; equivalently, its eigenvalue distribution $\{\mu_i^{(l)}\}$ is a valid probability mass function on B outcomes.

3 Matrix-based Rényi- α entropy

For $\alpha > 0$, $\alpha \neq 1$, define

$$H_\alpha(\hat{K}) = \frac{1}{1-\alpha} \log \sum_i \mu_i^\alpha = \frac{1}{1-\alpha} \log \text{tr}(\hat{K}^\alpha). \quad (3)$$

Special cases. $\alpha \rightarrow 1$ recovers the Shannon (von Neumann) entropy $H_1(\hat{K}) = -\sum \mu_i \log \mu_i$.

$\alpha = 2$ gives the **collision entropy**

$$H_2(\hat{K}) = -\log \sum_i \mu_i^2 = -\log \text{tr}(\hat{K}^2). \quad (4)$$

Equation (4) is the workhorse: it requires no eigendecomposition, scales as $O(B^2)$ from $\text{tr}(\hat{K}^2) = \sum_{i,j} \hat{K}_{ij}^2$, and is automatically differentiable in any standard autograd framework.

4 Joint distribution: Hadamard product

For two trace-1 PSD matrices $\hat{K}_i, \hat{K}_j \in \mathbb{R}^{B \times B}$, define the **joint matrix** as the entrywise (Hadamard) product, normalised:

$$\hat{K}_{ij} = (\hat{K}_i \odot \hat{K}_j) / \text{tr}(\hat{K}_i \odot \hat{K}_j). \quad (5)$$

Lemma 1 (Schur product theorem). $\hat{K}_i \odot \hat{K}_j$ is positive semi-definite.

Sketch. The Hadamard product of two PSD matrices is PSD; this is the classical Schur theorem. Both factors are PSD by construction (eq. 1). $\square$

Lemma 2 (Independence factorisation). *If $\hat{K}_i$ and $\hat{K}_j$ correspond to statistically independent features over the batch, then $H_\alpha(\hat{K}_{ij}) = H_\alpha(\hat{K}_i) + H_\alpha(\hat{K}_j)$.*

Sketch. Direct manipulation of $\text{tr}(\hat{K}^\alpha)$ using spectral-factor independence; full proof in Sanchez-Giraldo et al. (2014), Theorem 5. $\square$

5 Cross-layer mutual information

Define

$$I_\alpha(\hat{K}_i; \hat{K}_j) = H_\alpha(\hat{K}_i) + H_\alpha(\hat{K}_j) - H_\alpha(\hat{K}_{ij}). \quad (6)$$

Theorem 1 (Sanchez-Giraldo, Theorem 6). *For $\alpha \in (0, 1) \cup (1, \infty)$, $I_\alpha(\hat{K}_i; \hat{K}_j) \geq 0$, with equality iff $\hat{K}_{ij} = \hat{K}_i \odot \hat{K}_j$ holds in the independence-factorisation sense.*

Sketch. Apply the log-sum inequality to the spectrum of $\hat{K}_{ij}$ relative to the product of marginal spectra. The Hadamard product satisfies

$$\text{tr}((\hat{K}_i \odot \hat{K}_j)^\alpha) \leq \text{tr}(\hat{K}_i^\alpha)^{1/2} \cdot \text{tr}(\hat{K}_j^\alpha)^{1/2}$$

(Cauchy-Schwarz on traces); rearranging gives the bound. $\square$

Properties.

- Symmetry: $I_\alpha(\hat{K}_i; \hat{K}_j) = I_\alpha(\hat{K}_j; \hat{K}_i)$ — immediate from Hadamard commutativity.
- Boundedness: $0 \leq I_\alpha(\hat{K}_i; \hat{K}_j) \leq \min\{H_\alpha(\hat{K}_i), H_\alpha(\hat{K}_j)\}$.

6 Conditional entropy

Define

$$H_\alpha(\hat{K}_j \mid \hat{K}_i) = H_\alpha(\hat{K}_{ij}) - H_\alpha(\hat{K}_i). \quad (7)$$

Key properties.

- $H_\alpha(\hat{K}_j \mid \hat{K}_i) \geq 0$.
- $H_\alpha(\hat{K}_j \mid \hat{K}_i) = H_\alpha(\hat{K}_j) - I_\alpha(\hat{K}_i; \hat{K}_j)$ (chain rule).
- Chain rule recovers eq. (6).

7 Differentiability and computational complexity

Differentiability. With $\alpha = 2$,

$$H_2(\hat{K}) = -\log \text{tr}(\hat{K}^2), \quad \text{tr}(\hat{K}^2) = \sum_{i,j} \hat{K}_{ij}^2, \quad \hat{K} = (aa^\top) / \text{tr}(aa^\top).$$

Each operation is a smooth function of the activations a , hence of the weights through the chain rule. Standard autograd handles this without manual derivatives.

For $\alpha \neq 2$ we need eigenvalues; `torch.linalg.eigvalsh` provides gradients via the Bunch-Lehmberger formula on symmetric inputs.

Computational cost per regulariser-update step.

Step	Cost
Compute $\hat{K}_l$ for one layer	$O(B^2 d_l)$
Compute $H_2(\hat{K}_l)$ per layer	$O(B^2)$
Compute $\hat{K}_{ij}$ per pair	$O(B^2)$
Compute $H_2(\hat{K}_{ij})$ per pair	$O(B^2)$
Number of pairs	$L(L-1)/2$
Total at $\alpha = 2$	$O(L^2 B^2)$ per step

For our setups $B = 128$ and $L \in \{3, \dots, 21\}$ this is < 1 ms on GPU per regulariser update.

8 The four extensions, unified

Path E — temporal conditional entropy. For batch X shared across consecutive regulariser updates with layer- l Gram $\hat{K}_l^{(t)}$ and $\hat{K}_l^{(t-1)}$,

$$\mathcal{L}_E = \lambda \cdot H_\alpha(\hat{K}_l^{(t)} \mid \hat{K}_l^{(t-1)}).$$

Penalises *novel* layer information per step.

Path F — cross-layer mutual information.

$$\mathcal{L}_F = \lambda \cdot \sum_{i < j} I_\alpha(\hat{K}_i; \hat{K}_j).$$

Penalises redundancy between layers; encourages each layer to encode information not already in its predecessors.

Path G — joint view entropy. For dataflow- and factor-view adjacencies $K_{\text{df}}, K_{\text{ft}},$

$$\mathcal{L}_G = \lambda \cdot H_\alpha(\hat{K}_{\text{df}} \odot \hat{K}_{\text{ft}}).$$

Removes the user’s choice of view.

Path H — information bottleneck. For input/target batches with Gram $\hat{K}_X, \hat{K}_Y$ and a hidden representation $T = a_l(X)$ with Gram $\hat{K}_T,$

$$\mathcal{L}_H = \lambda \cdot I_\alpha(\hat{K}_X; \hat{K}_T) - \beta \cdot I_\alpha(\hat{K}_T; \hat{K}_Y).$$

The matrix-based information-bottleneck of Achille and Soatto.

Unification. Paths E, F, G, H are all functions of the same primitive $I_\alpha(\hat{K}_a; \hat{K}_b),$ with differing arguments (layer/time/view/data axes). Choosing one implementation gives all four extensions for free.

9 Caveats and design choices

1. **Data dependence.** Unlike the existing weight-only regularisers, $\hat{K}_l$ requires a forward pass. In our training loop this is free (one forward already happens for the task loss). *However,* the regulariser becomes stochastic across mini-batches.
2. **Mini-batch noise mitigation.** Sample a fixed held-out batch X_{reg} once at training start and compute $\hat{K}_l(X_{\text{reg}}).$ Removes mini-batch fluctuations.
3. **Choice of $\alpha.$** $\alpha = 2$ has the closed-form $-\log \text{tr}(\hat{K}^2)$ and is the recommended default for compute. Shannon ($\alpha \rightarrow 1$) is the “standard” entropy reviewers expect; we default to $\alpha = 2$ and report Shannon as a sensitivity analysis.
4. **Activation hook point.** Hook **post-linear, pre-activation** to keep the closest connection to the existing weight-side adjacency; the post-non-linear hook is also defensible and slightly cheaper.

10 Hypotheses for empirical testing (phase 8)

If layer redundancy is the structural cause of the residual negatives (circles, CapsMLP) under the original weight-side regulariser, we predict:

1. **Positive datasets** (spirals, MNIST plain MLP) should have non-negligible cross-layer MI; the MI penalty should preserve the positive sign — with possibly smaller magnitude than scalar entropy (since MI penalises a different structural quantity).
2. **Negative datasets** (circles, CapsMLP MNIST) should have *higher* cross-layer MI than positives — the failure mode is layers learning redundant information instead of a distributed code. If this prediction holds, decorrelating via MI penalty should *neutralise* the negatives more cleanly than scalar-entropy normalisation does.

3. **Variance reduction:** the dominant cross-cutting effect of the weight-side family was σ -reduction in val-acc across seeds. We predict the activation-side MI penalty also reduces σ , possibly *more strongly* on architectures with redundant pathways (CapsMLP, ResMLP-20).

A clear pass through these three predictions would justify presenting the activation-side framework as the principled refinement of the weight-side one in the manuscript.

References

- [1] Sanchez-Giraldo, L., Rao, M., and Principe, J. C. *Measures of Entropy from Data Using Infinitely Divisible Kernels*. IEEE Trans. Inform. Theory 61(1), 2014.
- [2] Yu, S., Sanchez-Giraldo, L., and Principe, J. C. *Multivariate Extension of Matrix-Based Rényi's α -Order Entropy Functional*. IEEE Trans. PAMI 42(8), 2019.
- [3] Wickstrøm et al. *Information Plane Analysis of Deep Neural Networks via Matrix-Based Rényi's Entropy and Tensor Kernels*. JMLR 21, 2020.
- [4] Achille, A. and Soatto, S. *Information Dropout: Learning Optimal Representations Through Noisy Computation*. IEEE Trans. PAMI 40(12), 2018.
- [5] Tishby, N. and Zaslavsky, N. *Deep Learning and the Information Bottleneck Principle*. ITW, 2015.